IMEN574 / IDSC574 · PROGRAMMING FOR AX

19

Language Models

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Language Models

01 **Contextual Tokens**

How tokens read context

02 **Training and Generation**

How objectives shape outputs

03 **Adapting and Verifying**

How evidence enters answers

01 · Contextual Tokens

01 **Contextual Tokens**
How tokens read context

02 **Training and Generation**
How objectives shape outputs

03 **Adapting and Verifying**
How evidence enters answers

Why Model Sequences

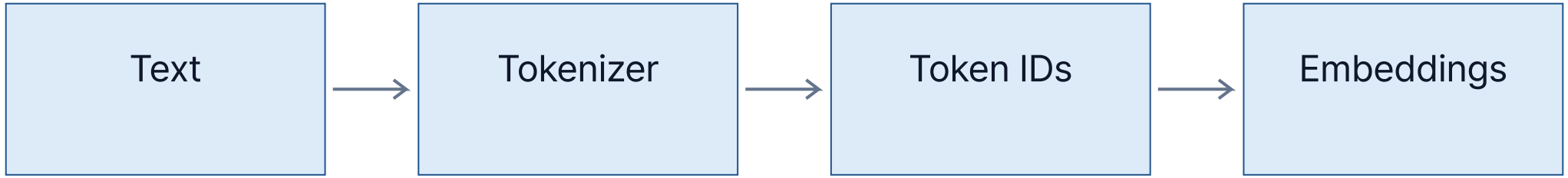
The same token can mean different things in context

Context A	river	bank
Context B	savings	bank

Context changes the information carried by a token

Tokens and Vocabulary

The model predicts token IDs rather than whole ideas



Text becomes an indexed sequence

Worked Example: Token Boundaries

Trace the inputs before execution

```
Text: the cats sat
Greedy vocabulary: cats, cat, sat, the, SPACE
Choose the longest listed token at each position; fallback to one character.
```

Demo: Token Boundaries

Executable educational calculation; no pretrained inference

Compare token rules for the cats sat; edit cats to cat and tokenize again

Text

```
the cats sat
```

Tokenize | Rule greedy

Token count = 5

[the] [␣] [cats] [␣] [sat]

Greedy vocabulary: cats, cat, sat, the, SPACE

Token lengths: 3, 1, 4, 1, 3.
Questions: Does “cat” equal “cats”? Can another vocabulary change the sequence?

Answer: Token Boundaries

Same inputs as the initial demo state

```
Greedy tokens [the][SPACE][cats][SPACE][sat], count 5
```

Failure: Token Boundaries

Identify the assumption that breaks

```
This fixed greedy tokenizer is educational, not BERT WordPiece
actual tokenizer IDs depend on model vocabulary
```

Embedding Lookup

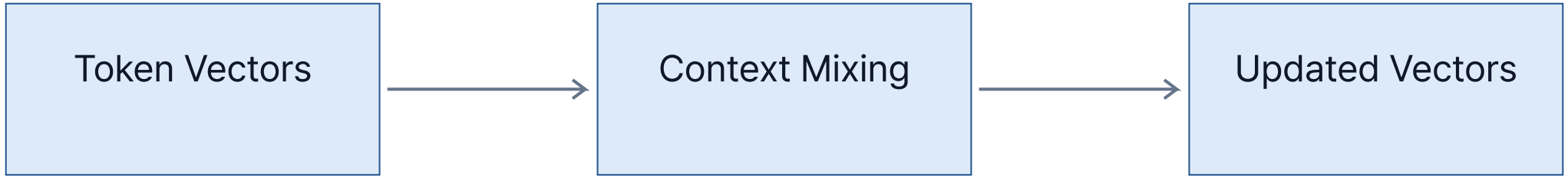
Token IDs select trainable vectors

	d1	d2
ID 0	0.2	0.4
ID 1	0.7	-0.1
ID 2	-0.3	0.9

An ID selects a row in the embedding table

Contextual Representations

A token vector changes after it reads other tokens

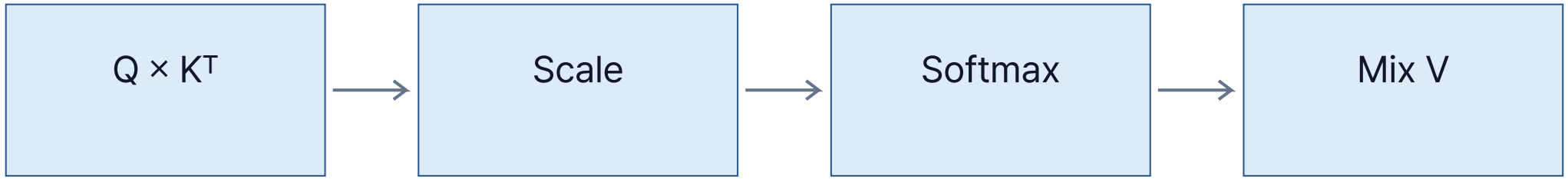


Context mixing changes the vector at each position

Scaled Dot-Product Attention

Queries choose a weighted mixture of values

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})V$$



Queries, Keys and Values

One input can supply three learned projections

	Query	Key	Value
Token 1	q1	k1	v1
Token 2	q2	k2	v2

Self-attention projects one sequence three ways

Attention Scores

Each query has a distribution across key positions

	key 1	key 2	key 3
query 1	0.7	0.2	0.1
query 2	0.1	0.8	0.1
query 3	0.2	0.3	0.5

Each row defines its own weighted mixture

Worked Example: Causal Attention

Trace the inputs before execution

```
Q = K = [[1,0],[0,1],[1,1]]
V = [[1,0],[0,2],[3,1]]; query index0
Scores = QKᵀ / √2; disallow key positions > query position.
```

Demo: Causal Attention

Executable educational calculation; no pretrained inference

Set future V to 99; compare the output with and without a causal mask

Query token 0 Mask Causal Future V[2,0] 3

Q = K = [[1,0], [0,1], [1,1]]

P(key 0)=1.000	P(key 1)=0.000	P(key 2)=0.000

softmax(QK^T / √2 + mask) V = [1.000, 0.000]

Raw scores: [0.707, 0.000, 0.707].
Questions: Can token 0 see token 2? Does changing future V alter the causal result?

Answer: Causal Attention

Same inputs as the initial demo state

```
Raw scores [.707107,0,.707107]
causal weights [1,0,0]
output [1,0]
```

Failure: Causal Attention

Identify the assumption that breaks

```
Mask future positions before softmax
changing only values must not change attention weights
```

Attention Masks

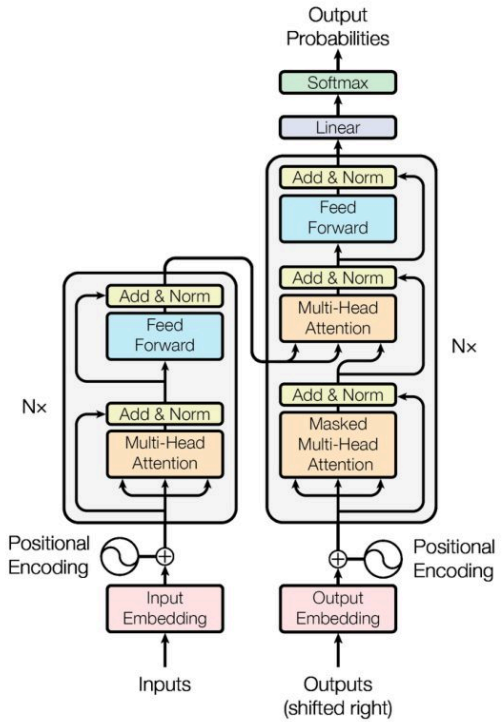
A mask excludes disallowed information paths

	key 1	key 2	key 3
position 1	1	0	0
position 2	1	1	0
position 3	1	1	1

Causal masking blocks future token access

Transformer Block

Attention is one component inside a residual block



The block combines mixing and per-token transformations

https://arxiv.org/abs/1706.03762

02 · Training and Generation

01 Contextual Tokens

How tokens read context

02 **Training and Generation**

How objectives shape outputs

03 Adapting and Verifying

How evidence enters answers

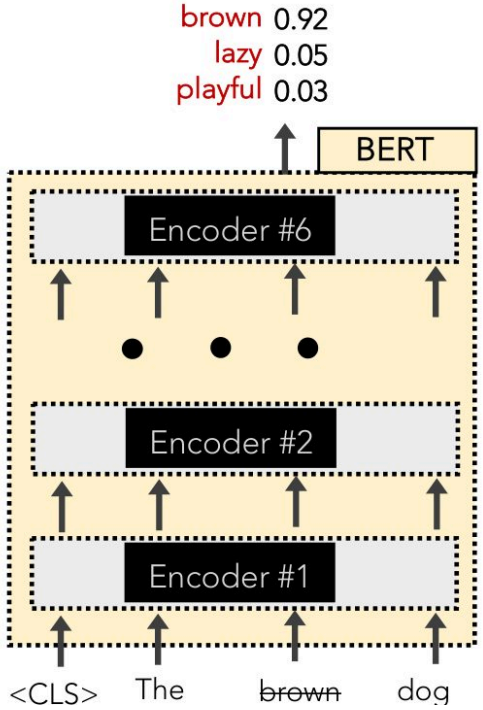
Why Two Training Objectives

Reading all tokens and predicting the next token need different masks

Objective	Visible context	Use
Masked token	Both directions	Representation
Next token	Prefix only	Generation

Masked Language Modeling

The model predicts hidden tokens from surrounding context

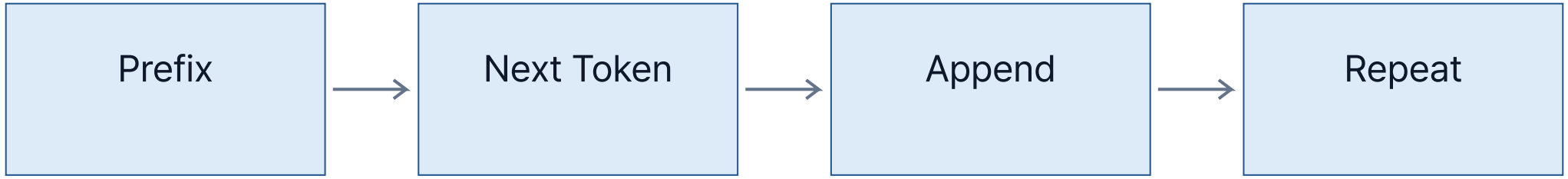


Both sides can inform a masked position

https://arxiv.org/abs/1810.04805

Autoregressive Factorization

A sequence probability is a product of conditional predictions



Each generated token becomes part of the next context

Next-Token Loss

Training compares each prefix with the observed next token

$$L = -\Sigma_t \log p(x_t \mid x_{<t})$$

Shifted Inputs and Targets

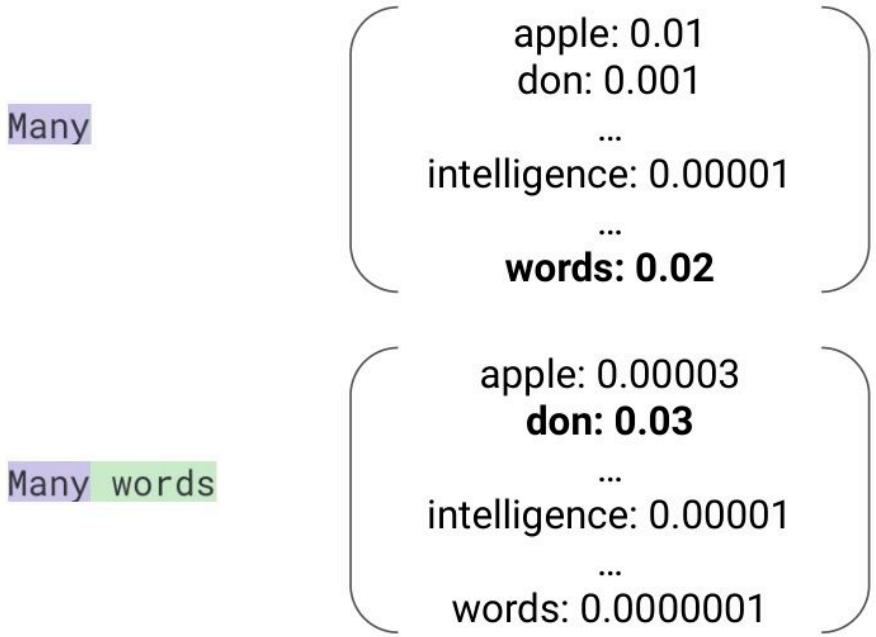
Every observed transition supplies a training example

	t1	t2	t3
Inputs	The	cat	sat
Targets	cat	sat	EOS

Targets are offset by one position

Token Generation

The next-token distribution grows one prefix at a time



A sampled token extends the prefix

https://arxiv.org/abs/2005.14165

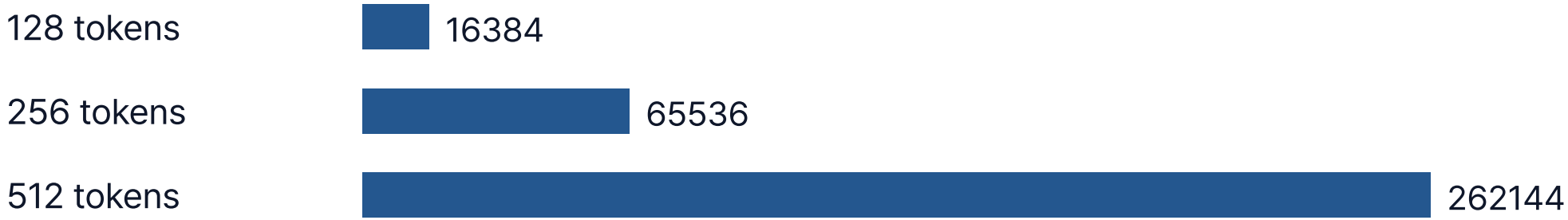
Decoding Policy

The same logits can produce different output sequences

Policy	Rule
Greedy	Choose the largest logit
Sampling	Draw from probabilities
Top-p	Restrict cumulative mass

Context Length and Cost

Longer context expands the token interaction matrix

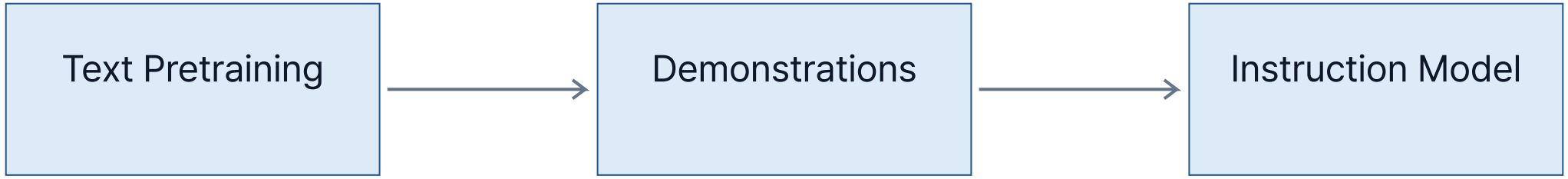


Dense attention score entries

Doubling sequence length quadruples score entries

Pretraining and Instruction Tuning

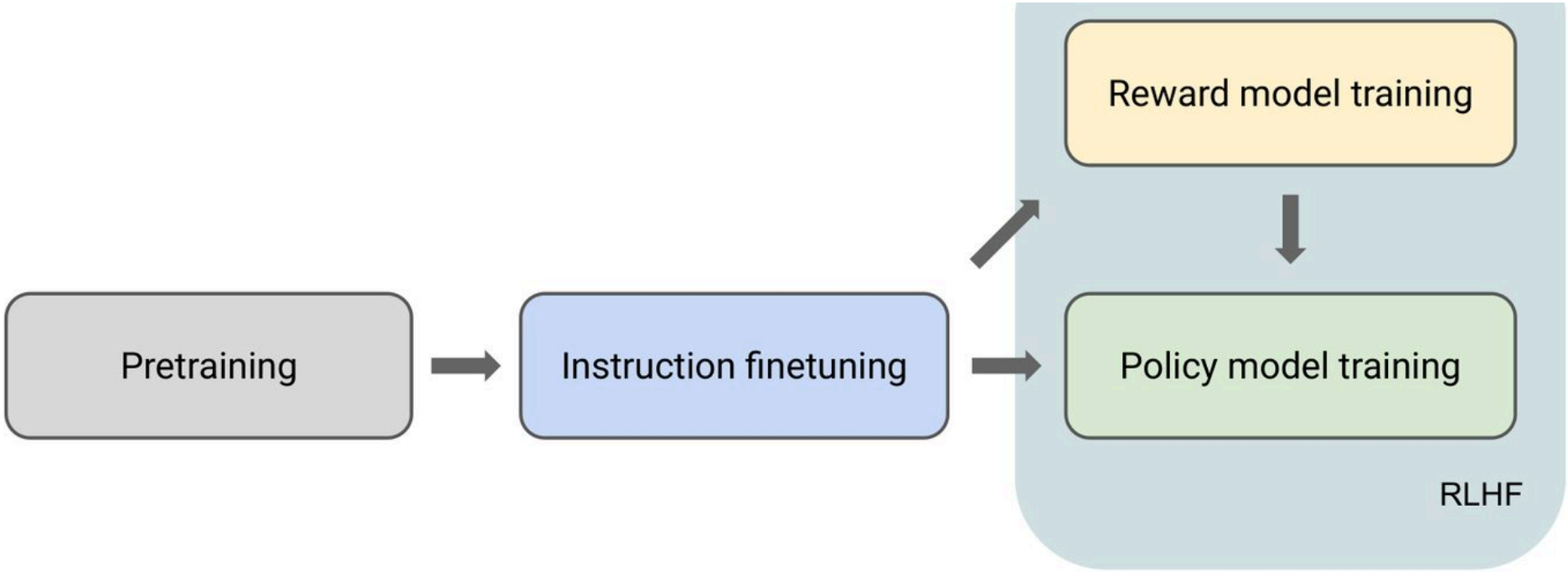
Following a request is trained after learning broad text patterns



Demonstrations teach the model a response format

Human Feedback Pipeline

Preference signals can further shape response behavior



Preference training changes which responses are favored

https://arxiv.org/abs/2203.02155

Caveat: Fluent Errors

Language likelihood does not certify factual truth

Item	Field 1	Field 2
Surface	Plausible wording	high
Evidence	Source support	absent

A likely continuation can lack supporting evidence

03 · Adapting and Verifying

01 Contextual Tokens

How tokens read context

02 Training and Generation

How objectives shape outputs

03 **Adapting and Verifying**

How evidence enters answers

Why Adaptation Methods Differ

A model may need knowledge, behavior or examples

Need	Mechanism
New document evidence	Retrieval
A response format	Context examples
Persistent adaptation	Parameter updates

In-Context Learning

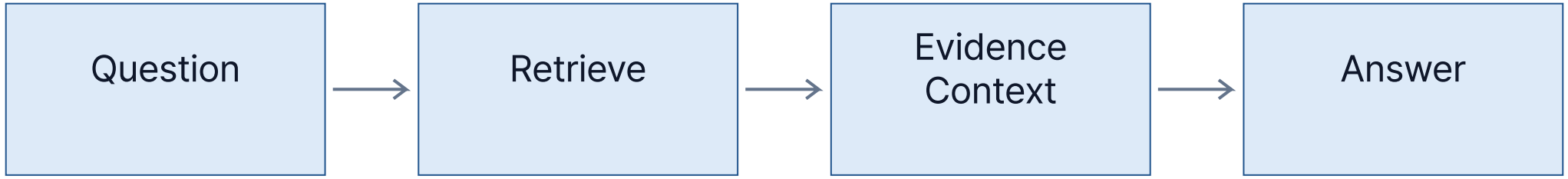
Examples change the context while weights stay fixed

Item	Field 1	Field 2
Examples	great → positive	poor → negative
Query	fine → ?	weights unchanged

Context examples change the conditional prediction

Retrieval-Augmented Generation

Retrieved text supplies evidence for a current question



Evidence enters through the prompt context

Retrieval Ranking

The chosen documents determine the available evidence



Illustrative retrieval similarity

High similarity is a ranking signal, not proof

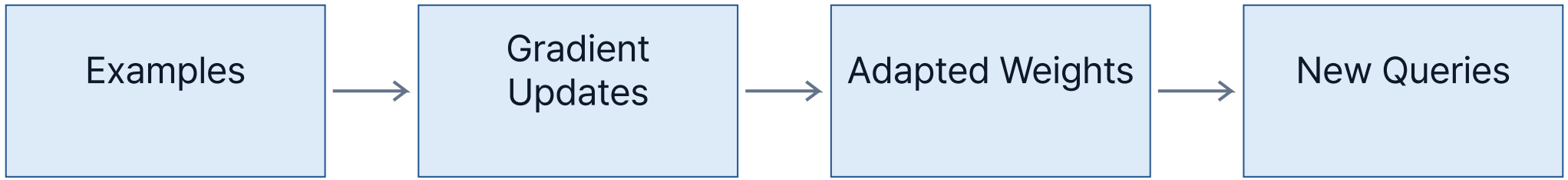
Grounded Answer Contract

An answer must retain links to the supporting passages

Claim	Evidence ID
Valve opened at 10:05	Log 17
Pressure fell later	Sensor 9

Fine-Tuning Scope

Parameter updates persist beyond a single prompt



Fine-tuning changes the stored model parameters

Structured Output Validation

Parseable output still needs semantic checks

```
answer = {"count": 3, "unit": "items"}
assert isinstance(answer["count"], int)
assert answer["count"] >= 0
assert answer["unit"] == "items"
```

Prompt Evaluation Set

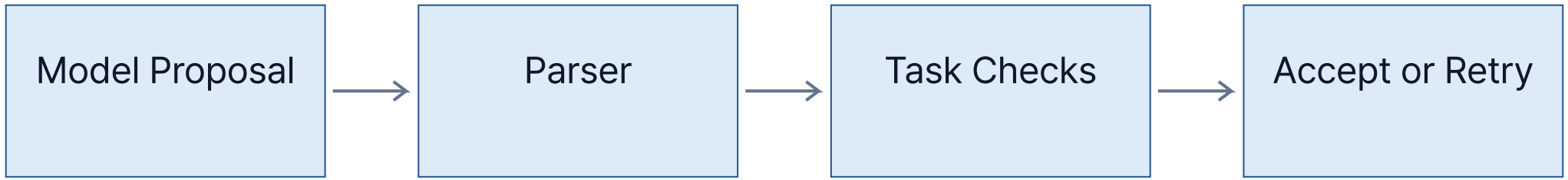
Keep examples that expose known failure modes

	Prompt A	Prompt B	Prompt C
Simple	1	1	0
Ambiguous	1	0	1
Adversarial	0	1	1

Different prompts fail on different input slices

Model and Harness Boundary

Generation supplies a proposal while software checks the result



A software check decides whether to use a proposal

Language Model Audit

Record context, weights and decoding as separate choices

Component	Record
Context	Prompt and retrieved passages
Weights	Model and adaptation
Decoding	Sampling and stop rules

Run the Original Model Lab

Actual pretrained execution remains a separate, explicit runtime path

Original cells (0-based)	Model workflow	Observe
Original19:7–18	BERT tokenizer → embeddings → masked-token predictions	Inspect actual tokens and IDs; compare contextual meanings.
Original20:7–12	Causal LM next-token logits → temperature and sampling	Run actual model logits in the preserved original notebook.

Original model code remains available through the original-lab panel

Takeaway: Language Models

01 A language model predicts conditioned token sequences

02 Context, weights and decoding are different controls

03 Fluent output still requires task-specific evidence